Árboles de decisión para regresión y evaluación de modelos

Sesión 7 · Módulo 2: Modelos de regresión

Árboles CART · Poda · Validación cruzada · Sesgo–varianza de los modelos de regresión

Datos simulados · scikit-learn

Plan de la sesión (≈50 min)

Qué veremos

- **A.** Árboles de decisión para regresión: el algoritmo CART y la impureza (MSE)
- **B.** Poda: controlar el tamaño con `max_depth` y `min_samples_leaf`
- **C.** Validación cruzada: k pliegues, estratificada y anidada
- **D.** Análisis sesgo–varianza de **todos** los modelos de regresión del curso

Dos hilos conductores: un modelo **muy fácil de leer** (el árbol) y una forma **honesta de medir** cualquier modelo (la validación cruzada).

De dónde venimos (Módulo 2)

Los modelos de regresión que ya conocemos

Sesión	Modelo y su control
4	Lineal y polinomial: **grado**
5	Ridge, Lasso, Elastic Net: α, `l1_ratio`
6	SVR: C, ε, **kernel**
7 (hoy)	**Árbol de decisión**: profundidad, tamaño de hoja

Hoy cerramos el módulo

Todos los modelos tienen un control que regula su **complejidad**. Hoy aprendemos **el último modelo**, **cómo elegir bien esos controles** y **cómo compararlos todos** con la lente del sesgo y la varianza.

Después de esta sesión haremos el repaso de medio curso y el informe comparativo.

A1 · La idea: hacer preguntas de sí o no

Como el juego de las veinte preguntas

Para predecir el precio de una casa, el árbol pregunta:

- ¿La superficie es **menor de 106 m²**?
- Si sí: ¿la casa tiene **menos de 21 años**?
- . . . hasta llegar a una **hoja**.

Predicción = el **promedio** de los precios de las casas de entrenamiento que cayeron en esa hoja.

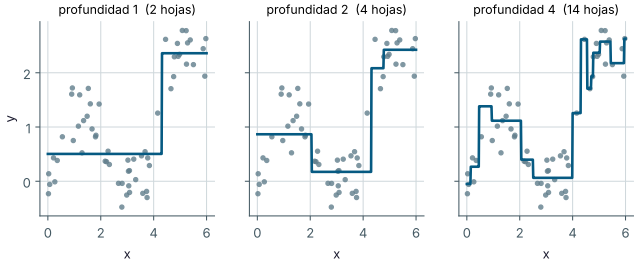
Vocabulario

- **Nodo**: una pregunta ("¿$x_j \leq s$?").
- **Raíz**: la primera pregunta.
- **Hoja**: nodo final; guarda un promedio.
- **Profundidad**: número de preguntas en el camino más largo.

Por qué gustan

Se **leen** como reglas, no necesitan escalar variables y capturan **no linealidades** sin que las diseñemos.

A2 · Un árbol aproxima una curva con escalones



Lectura

- **Profundidad 1**: una sola pregunta, dos escalones. Muy tosco.
- **Profundidad 2 y 4**: más hojas, más escalones, mejor seguimiento de la curva.
- Cada escalón es el **promedio** de los puntos de su hoja.

Idea

Más profundidad = más hojas = más flexibilidad.
¿Les suena a "grado del polinomio"?

A3 · La impureza: ¿qué tan “revuelta” está una hoja?

Impureza de un nodo en regresión: el MSE

$$\mathrm{MSE}(\text{nodo}) = \frac{1}{n_{\text{nodo}}} \sum_{i \in \text{nodo}} \left(y_i - \bar{y}_{\text{nodo}}\right)^2$$

Es la **varianza** de los y del nodo respecto a su promedio $\bar{y}$. Como predecimos con $\bar{y}$, esa varianza es justamente el **error de entrenamiento** del nodo.

Cómo se lee

- Hoja con y casi iguales: MSE ≈ 0 (“pura”).
- Hoja con y muy dispersos: MSE alto (“revuelta”).

Objetivo de cada pregunta

Dividir un nodo en dos hijos de modo que la **impureza ponderada baje lo más posible**:

$$\frac{n_I}{n}\,\mathrm{MSE}_I + \frac{n_D}{n}\,\mathrm{MSE}_D$$

I = hijo izquierdo, D = hijo derecho; cada uno pesa según cuántos datos recibe.

A4 · El algoritmo CART, paso a paso

CART (*Classification And Regression Trees*)

1. Empezar con **todos** los datos en un nodo (la raíz).
2. Para **cada variable** x_j y **cada corte posible** s (puntos medios entre valores consecutivos): calcular la impureza ponderada de $\{x_j \leq s\}$ y $\{x_j > s\}$.
3. Quedarse con el **mejor par** (j, s) y partir el nodo.
4. **Repetir** lo mismo, por separado, en cada hijo.
5. Parar cuando se cumpla una **regla de paro** (profundidad máxima, pocas muestras...) y guardar el promedio en la hoja.

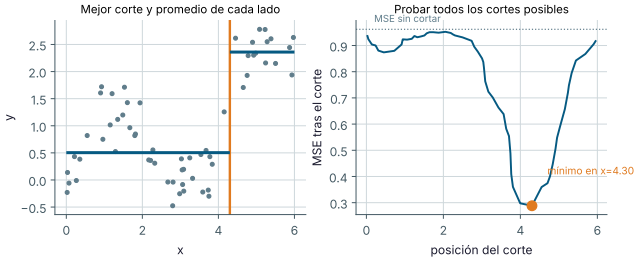
Dos palabras clave

- **Binario**: cada pregunta da exactamente dos ramas.
- **Voraz** (*greedy*): en cada nodo escoge el corte que **más baja la impureza en ese instante**, y **nunca regresa** a cambiar un corte ya hecho, aunque eso le cierre el paso a una mejor combinación de cortes más adelante.

Por qué voraz

Buscar el árbol óptimo entre todos los árboles posibles es computacionalmente inviable; el enfoque voraz es rápido y funciona bien en la práctica.

A5 · Un corte, visto en datos: probamos todos y elegimos el mínimo



Lectura

- Izquierda: el mejor corte divide los datos en dos grupos y cada uno se predice con su promedio.
- Derecha: para cada posición del corte, el MSE ponderado. Sin cortar es 0.96; el mejor corte (en $x = 4.30$) lo baja a **0.29**.
- Después **repetimos** en cada lado.

A6 · Ejemplo a mano con 6 puntos: calculando el MSE

Datos

x	1	2	3	4	5	6
y	2.0	2.4	2.2	5.1	4.9	5.4

Promedio global $\bar{y} = \frac{22.0}{6} = 3.67$.

$$\mathrm{MSE}_{\text{raíz}} = \tfrac{1}{6}\sum_i (y_i - \bar{y})^2 = \tfrac{13.11}{6} = 2.19$$

El corte ganador: $x \leq 3.5$, paso a paso

Izq. $\{2.0, 2.4, 2.2\} \rightarrow$ prom. 2.20:

$$\mathrm{MSE}_I = \tfrac{1}{3}\left[(2.0-2.2)^2+(2.4-2.2)^2+(2.2-2.2)^2\right] = 0.027$$

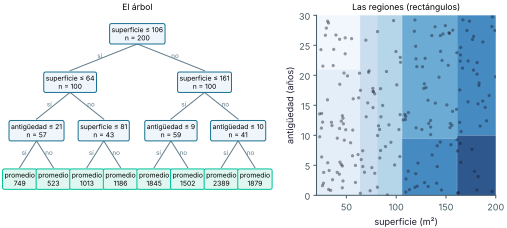
Der. $\{5.1, 4.9, 5.4\} \rightarrow$ prom. 5.13, $\mathrm{MSE}_D = 0.042$ (mismo cálculo).
Ponderado: $\frac{3}{6}(0.027) + \frac{3}{6}(0.042) =$ **0.034**. Las hojas predicen 2.2 y 5.1.

Cortes candidatos

Corte	Prom. izq.	Prom. der.	MSE ponderado
$x \leq 1.5$	2.00	4.00	1.630
$x \leq 2.5$	2.20	4.40	1.110
$x \leq 3.5$	2.20	5.13	0.034
$x \leq 4.5$	2.92	5.15	1.085
$x \leq 5.5$	3.32	5.40	1.585

MSE ponderado $= \frac{n_I}{6}\,\mathrm{MSE}_I + \frac{n_D}{6}\,\mathrm{MSE}_D$. El de menor valor gana.

A7 · Con varias variables: el árbol corta el espacio en rectángulos



Datos simulados de precios de viviendas (200 casas). En cada nodo CART **elige también la variable**: aquí la raíz pregunta por la **superficie** (corte en 106 m²). Cada hoja es un **rectángulo** del plano con un precio constante.

El MSE se calcula igual; lo que cambia

Con p variables, el paso 2 de CART (buscar el mejor corte) se repite **una vez por cada variable** x_j, con la misma fórmula del MSE. Al final se comparan los mejores cortes entre sí.

Ejemplo con 4 puntos y 2 variables

	1	2	3	4
x_1	1	2	3	4
x_2	5	1	4	2
y	1	1	5	5

Mejor corte en x_1: $x_1 \leq 2.5 \Rightarrow$ MSE pond. $= \mathbf{0.000}$.
Mejor corte en x_2: $x_2 \leq 1.5 \Rightarrow$ MSE pond. $= 2.667$.
Se compara el mejor corte **de cada variable**; gana el par (j, s) con menor MSE global: aquí $(x_1,\ 2.5)$.

A8 · En scikit-learn: DecisionTreeRegressor

```python
from sklearn.tree import DecisionTreeRegressor,
    export_text

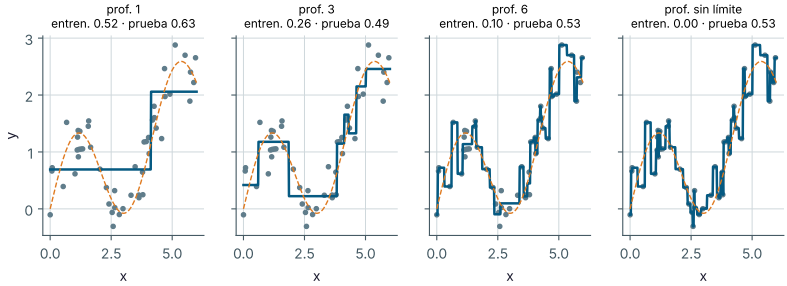
arbol = DecisionTreeRegressor(
    criterion="squared_error",    # impureza = MSE
    max_depth=3,
    random_state=0)
arbol.fit(Xtr, ytr)

print(export_text(arbol,
      feature_names=["superficie", "antiguedad"]))
print(arbol.get_n_leaves(), arbol.get_depth())
print(arbol.predict(Xte[:3]))
```

Puntos clave

- `criterion="squared_error"` es el MSE de hoy (es el valor por defecto).
- `export_text` imprime las **reglas**; `plot_tree` las dibuja.
- **No hace falta escalar** las variables: solo se comparan valores con un corte.
- Sin límites, el árbol crece hasta que cada hoja es pura (**sobreajuste**): lo vemos enseguida.

B1 · Sin límites, el árbol memoriza



Lectura

- Profundidad 1: **subajuste** (error alto en entrenamiento y en prueba).
- Sin límite: cada punto tiene su hoja (50 hojas), error de entrenamiento **0.00**... pero la prueba no mejora (0.53).

Conclusión

Un árbol sin freno **siempre** llega a error de entrenamiento 0: es sobreajuste (varianza alta). Hay que **limitar su crecimiento**; a eso se le llama **poda**.

B2 · Dos tipos de poda

Pre-poda (parar antes)

Se detiene el crecimiento **mientras** se construye el árbol. Es lo que usaremos hoy:

- `max_depth`: profundidad máxima.
- `min_samples_leaf`: mínimo de datos por hoja.
- `min_samples_split`: mínimo de datos para poder partir un nodo.
- `max_leaf_nodes`: tope de hojas.

Post-poda (podar después)

Se deja crecer el árbol completo y luego se **recortan** ramas que no compensan. En scikit-learn: `ccp_alpha` (poda por costo–complejidad), parecido al α de Ridge y Lasso: *error* $+$ $\alpha\times$ *número de hojas*.

Idea común

Todas son el **control de complejidad** del árbol: **menos hojas** $\Rightarrow$ menos varianza y más sesgo.

B3 · Poda por costo–complejidad: el parámetro α

La idea: penalizar árboles grandes

$$R_\alpha(T) = \underbrace{R(T)}_{\text{error total}} + \alpha \underbrace{|T|}_{\#\text{ hojas}}$$

$R(T)$ = suma del SSE de todas las hojas; $|T|$ = número de hojas. $\alpha \geq 0$ penaliza tener más hojas, igual que el α de Ridge/Lasso penaliza coeficientes grandes.

El algoritmo (poda del "enlace más débil")

1. Crecer el árbol **completo** T_0 (sin límites).
2. Para cada nodo interno t: si se colapsara su subárbol T_t a una sola hoja, ¿qué tan "caro" sería? $\alpha_{\text{ef}}(t) = \dfrac{R(t) - R(T_t)}{|T_t| - 1}$. ($R(t)$: t tratado como una sola hoja; $R(T_t)$: el subárbol completo bajo t, sin colapsar.)
3. Podar el nodo con el α_{ef} **más pequeño** (el más fácil de sacrificar). Repetir sobre el árbol ya podado.
4. Esto genera una secuencia $T_0 \supset T_1 \supset \cdots \supset \{\text{raíz}\}$, cada uno válido para un rango de α.

Ejemplo numérico

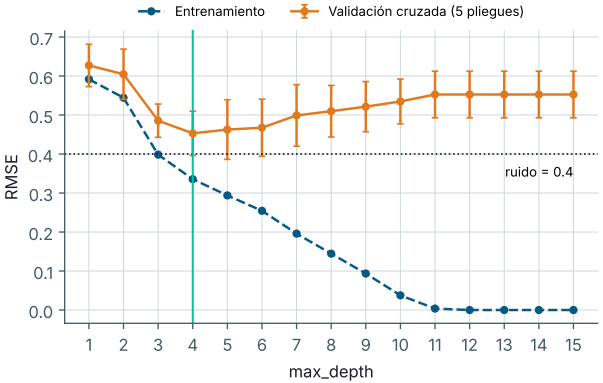
Un subárbol con 2 hojas tiene error combinado $R(T_t) = 0.5$. Si se colapsa a 1 sola hoja, el error sube a $R(t) = 0.8$.

$$\alpha_{\text{ef}} = \frac{0.8 - 0.5}{2 - 1} = 0.3$$

Lectura: si $\alpha \geq 0.3$, ya no compensa mantener esas 2 hojas (el ahorro en error no paga el costo de la hoja extra) y se podan; si $\alpha < 0.3$, sí compensa y se conservan.

α no se adivina: se prueba cada árbol de la secuencia con **validación cruzada**, igual que con α en Ridge/Lasso. En scikit-learn: `DecisionTreeRegressor(ccp_alpha=...)`; `cost_complexity_pruning_path(X,y)` devuelve toda la secuencia.

B4 · `max_depth`: elegirlo con validación



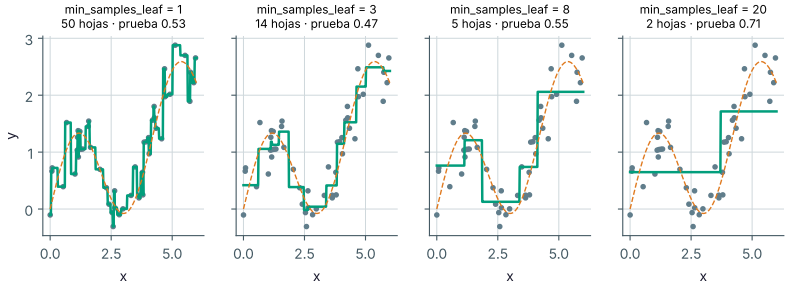
Lectura

- **Entrenamiento**: baja siempre; llega a 0.
- **Validación**: forma de "U", mínimo en profundidad **4** (RMSE 0.45, ya cerca del ruido 0.4).
- Con profundidad 15 la validación sube a 0.55: sobreajuste.

Es la misma "U" de las Sesiones 4 y 5

Ahora el eje horizontal es la **profundidad**: más profundo = más complejo.

B5 · `min_samples_leaf`: exigir que cada hoja tenga “testigos”



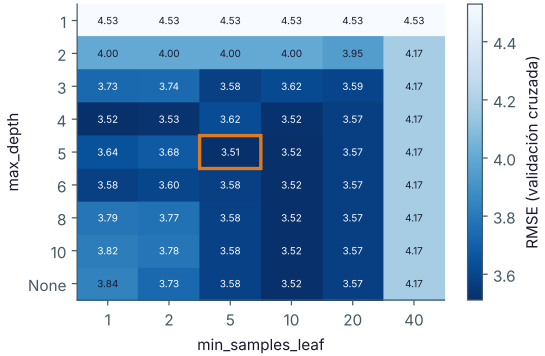
Lectura

- `min_samples_leaf`$= 1$: una hoja por punto, sigue el ruido.
- Con 3: curva más suave y la **mejor prueba** (0.47 frente a 0.53 con 1).
- Con 8 ya empieza a faltar flexibilidad (0.55) y con 20 hay solo 2 hojas: subajuste (0.71).

Intuición

Promediar **más puntos por hoja** da predicciones más estables: el ruido de unos se cancela con el de otros.

B6 · Se eligen juntos: búsqueda en malla



El experimento

Datos simulados de Friedman (10 variables, 5 útiles), 210 de entrenamiento. Cada celda: RMSE de validación cruzada (5 pliegues) para una pareja `max_depth`–`min_samples_leaf`.

Lectura

- Mejor: profundidad 5 y hoja mínima 5 (RMSE 3.51).
- Árbol sin podar (`None`, 1): 3.84.
- Zona de buenos valores **amplia**: no hace falta afinar al decimal.
- En prueba: 3.42 con el árbol podado, frente a 3.54 sin podar.

B7 · En scikit-learn: `GridSearchCV` sobre el árbol

```python
from sklearn.model_selection import GridSearchCV, KFold

malla = {"max_depth": [2, 3, 4, 5, 6, 8, None],
         "min_samples_leaf": [1, 2, 5, 10, 20]}

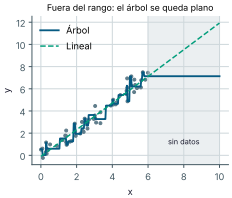
busqueda = GridSearchCV(
    DecisionTreeRegressor(random_state=0), malla,
    cv=KFold(5, shuffle=True, random_state=0),
    scoring="neg_root_mean_squared_error")
busqueda.fit(Xtr, ytr)

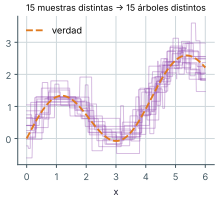
print(busqueda.best_params_)
print(-busqueda.best_score_)      # RMSE de validacion
mejor = busqueda.best_estimator_  # ya reentrenado
```

Puntos clave

- `scoring` es un puntaje: **mayor es mejor**; por eso el RMSE aparece **negativo**.
- `GridSearchCV` prueba cada combinación con validación cruzada y reentrena la mejor con **todo** el entrenamiento.
- El conjunto de **prueba** no se toca hasta el final.

B8 · Ventajas y límites de los árboles





A favor

- Interpretables: reglas *si–entonces*.
- Sin escalado; toleran variables de escalas muy distintas.
- Capturan no linealidades e interacciones.

En contra

- **No extrapolan**: fuera del rango, predicen un valor constante.
- **Inestables**: otra muestra, otro árbol (varianza alta).
- Predicción escalonada, no suave.

Para no olvidar

La inestabilidad es el punto débil de un árbol solo. Se arregla **promediando muchos árboles** (bagging y bosques aleatorios, Sesión 13).

Pausa para pensar (actividad 2, 3 min)

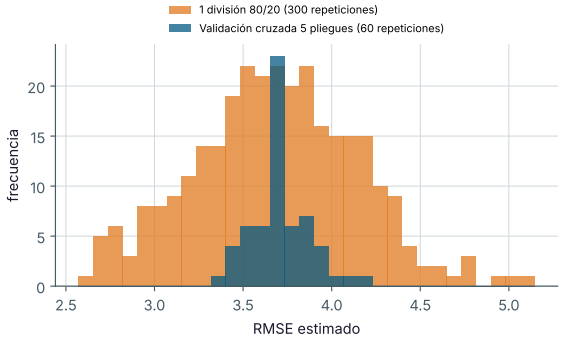
Trabaja de forma individual

Un árbol sin límites obtiene RMSE de **0.0** en entrenamiento y **3.5** en validación.

1. ¿Sesgo alto o varianza alta? ¿Por qué?
2. ¿Subirías o bajarías `max_depth`? ¿Y `min_samples_leaf`?
3. Tienes datos de x entre 0 y 100 y debes predecir para $x = 150$. ¿Por qué un árbol es mala idea? ¿Qué modelo del curso usarías?

Respuestas: (1) varianza alta: memoriza el entrenamiento. (2) Bajar `max_depth` y subir `min_samples_leaf`. (3) El árbol no extrapola: predice el valor de la última hoja; un modelo lineal (o polinomial de grado bajo, o Ridge) sí extiende la tendencia.

C1 · Una sola división entrenamiento/validación no basta



¿Por qué es un problema?

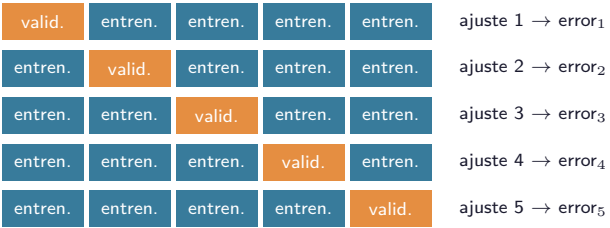
Partir 80/20 una sola vez es, en el fondo, **un sorteo**: qué filas caen en el 20% de validación depende de la semilla aleatoria. Con pocos datos, ese 20% puede tocar por casualidad puntos "fáciles" (RMSE bajo) o "difíciles" (RMSE alto) — y el número que reportamos cambia mucho según qué nos tocó.

El experimento

Mismos 150 datos y mismo árbol. Repetimos **300 veces** la división 80/20, cambiando sólo la semilla, y anotamos el RMSE de validación cada vez.

C2 · Validación cruzada de k pliegues

Los datos de entrenamiento, partidos en $k = 5$ pliegues



Procedimiento

1. Barajar y partir los datos en k pliegues.
2. Para cada pliegue: entrenar con los otros $k-1$ y medir el error en él.
3. **Promediar** los k errores (y mirar su dispersión).

Qué consigue

Todos los datos se usan para validar **una vez** y para entrenar $k-1$ veces. El estimado es más estable y el costo es de k **ajustes**.

C4 · Elegir k: una tabla de decisión

Compromiso

k	Sesgo	Costo
2–3	alto (pesimista)	2–3 ajustes
5 o 10	**bajo**	**5–10 ajustes**
n (*leave-one-out*)	el menor	n ajustes

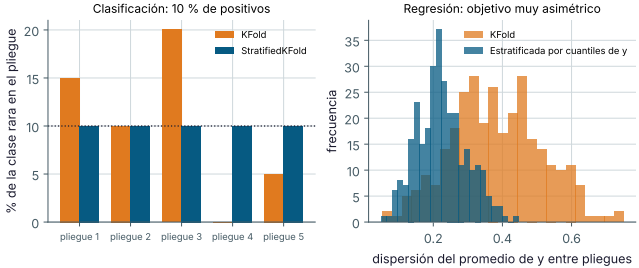
Recomendación

$k = 5$ si el modelo es costoso; $k = 10$ si hay tiempo. *Leave-one-out* solo con pocos datos y modelos baratos.

Dos hábitos

- Usar `shuffle=True` y `random_state` fijo: reproducible y sin orden oculto en los datos.
- Reportar **media** $\pm$ **desviación** de los k errores, no solo la media.

C5 · Validación cruzada estratificada



Idea

Estratificar = que cada pliegue conserve la **proporción** de cada grupo del conjunto completo.

Lectura

- Izquierda (clasificación, 10 % de positivos): `KFold` dejó pliegues con 0 % y 20 %; `StratifiedKFold`, **10 % en todos**.
- Derecha (regresión): con y muy asimétrico, estratificar por **cuantiles de** y reduce la dispersión del promedio entre pliegues (0.22 vs 0.38).

C6 · Estratificada en scikit-learn (y en regresión)

```python
import pandas as pd
from sklearn.model_selection import StratifiedKFold

# Clasificacion: estratifica por la clase
skf = StratifiedKFold(5, shuffle=True, random_state=0)
for tr, va in skf.split(X, y_clase):
    ...

# Regresion: estratifica por cuantiles de y
grupos = pd.qcut(y, q=5, labels=False)
for tr, va in skf.split(X, grupos):
    ...
```

Puntos clave

- En **clasificación** con clases desbalanceadas, es la opción por defecto (`cross_val_score` la usa solo en clasificadores).
- En **regresión** hay que dar los grupos a mano; vale la pena con y muy sesgado o pocos datos.
- Para series de tiempo o datos agrupados: `TimeSeriesSplit`, `GroupKFold`.

C7 · El problema de elegir y evaluar con los mismos datos

La trampa

Probamos **muchas** combinaciones de hiperparámetros con validación cruzada y reportamos la **mejor puntuación**. Pero esa puntuación la **elegimos por ser la mejor**: parte es suerte.

Como un examen con 60 intentos

Si cada intento tiene algo de azar y nos quedamos con la **calificación más alta**, la nota estará inflada. Cuantas más combinaciones y menos datos, **más optimismo**.

La solución: validación cruzada anidada

- **Bucle interno**: elige los hiperparámetros (`GridSearchCV`).
- **Bucle externo**: evalúa **todo el procedimiento**, incluida la búsqueda, en datos que la búsqueda **nunca vio**.

El externo no estima "el mejor modelo", sino **qué tan bien funciona "elegir bien y entrenar"**.

C8 · Validación cruzada anidada: el esquema

Bucle externo (evaluación): 5 pliegues



el pliegue de prueba queda **apartado**

Bucle interno (selección), solo con los 4 pliegues de entrenamiento:

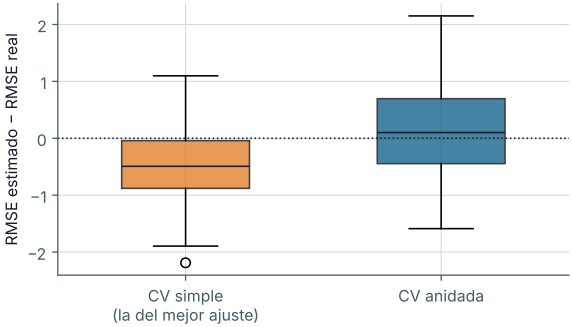


`GridSearchCV`: prueba cada combinación, elige la mejor y la reentrena

el modelo elegido se mide en el pliegue **apartado**

Resultado: promedio de los 5 errores externos = estimado honesto de todo el procedimiento

C9 · ¿Cuánto optimismo hay? Simple frente a anidada



El experimento

100 repeticiones con 50 datos ruidosos y una malla de 63 combinaciones de `max_depth` y `min_samples_leaf`. Se compara cada estimado con el error **real** en 20 000 datos nuevos.

Lectura

- **CV simple**: sesgo medio de **-0.50** (negativo = **optimista**).
- **CV anidada**: sesgo medio de **0.18**.

Mensaje

La CV simple es **optimista** (0.50 por debajo del error real); la anidada queda mucho más cerca (0.18, algo pesimista porque entrena con menos datos). Con más combinaciones y menos datos, la brecha crece.

C10 · En scikit-learn: validación cruzada anidada

```python
from sklearn.model_selection import cross_val_score

interno = KFold(5, shuffle=True, random_state=1)
externo = KFold(5, shuffle=True, random_state=2)

busqueda = GridSearchCV(
    DecisionTreeRegressor(random_state=0), malla,
    cv=interno,
    scoring="neg_root_mean_squared_error")

# el GridSearchCV completo es "el modelo" del bucle externo
puntajes = cross_val_score(
    busqueda, X, y, cv=externo,
    scoring="neg_root_mean_squared_error")

print(-puntajes.mean(), puntajes.std())
```

Puntos clave

- `GridSearchCV` se pasa **como estimador** a `cross_val_score`.
- Cuesta $k_{\text{ext}} \times k_{\text{int}} \times |\text{malla}|$ ajustes: reducir la malla o usar `n_jobs=-1`.
- Sirve para **comparar modelos** (cada uno con su malla), no para escoger los hiperparámetros finales.

C11 · Errores clásicos al validar

Qué evitar

- **Fuga de datos**: escalar o imputar **antes** de dividir. Todo preprocesamiento va **dentro del** `Pipeline`.
- **Mirar la prueba** para decidir (ni siquiera "una miradita").
- Reportar la **mejor** puntuación de la búsqueda como si fuera el error esperado.
- Ignorar el orden: en series de tiempo, no se baraja.

Receta recomendada

1. Separar un conjunto de **prueba** y no tocarlo.
2. Con el entrenamiento: `Pipeline` + `GridSearchCV`.
3. Para **comparar** modelos: validación cruzada **anidada**.
4. Entrenar el elegido con todo el entrenamiento.
5. Medir en la prueba **una sola vez**.

Pausa para pensar (actividad 3, 3 min)

Trabaja de forma individual

1. Probaste 200 combinaciones de hiperparámetros con validación cruzada de 5 pliegues y el mejor RMSE fue 2.1. ¿Es un estimado honesto del error en datos nuevos? ¿Cómo lo corregirías?
2. Un conjunto con 4 % de casos positivos. ¿Qué validación usas y por qué?
3. ¿Cuántos ajustes cuesta una validación anidada de 5×5 con una malla de 20 combinaciones?

Respuestas: (1) No, es optimista: la combinación se escogió por ser la mejor; usar validación cruzada anidada (o un conjunto de prueba apartado). (2) Estratificada: cada pliegue conserva el 4 %. (3) $5 \times (5 \times 20) + 5 = 505$ ajustes (los 5 finales reentrenan la mejor combinación en cada pliegue externo).

D1 · Repaso: sesgo, varianza y ruido

Las tres fuentes de error

$$\text{Error esperado} = \underbrace{\text{Sesgo}^2}_{\text{modelo muy rígido}} + \underbrace{\text{Varianza}}_{\text{modelo muy sensible}} + \underbrace{\text{Ruido}}_{\text{inevitabl}}$$

- **Sesgo**: error sistemático; el modelo **no puede** representar la verdad.
- **Varianza**: cuánto **cambia** el modelo si cambian los datos de entrenamiento.

¿Cómo se mide?

En la práctica solo tenemos **una** muestra. Para *verlas* hay que **simular**: conocer la función verdadera, generar **muchos** conjuntos de entrenamiento, ajustar el modelo en cada uno y comparar sus predicciones.

Sesgo^2 = qué tan lejos queda el *promedio* de los modelos de la verdad. Varianza = qué tanto se *dispersan* los modelos alrededor de su promedio.

D2 · El experimento: ocho modelos, 150 muestras de entrenamiento

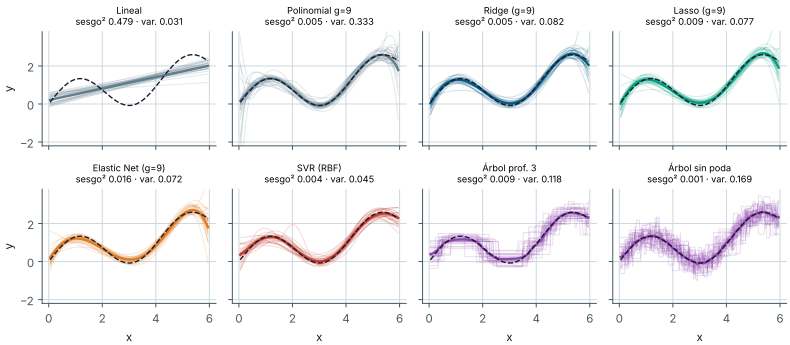
Montaje

- Verdad: $f(x) = \sin(1.5x) + 0.3x$ en $[0, 6]$; ruido $\sigma = 0.4$ (varianza del ruido $= 0.16$).
- Cada muestra: **40 puntos**. Repetimos **150 veces** con datos nuevos.
- Cada modelo **elige sus hiperparámetros por validación cruzada** dentro de cada repetición (salvo el polinomio y los árboles, fijos a propósito).

Los ocho modelos

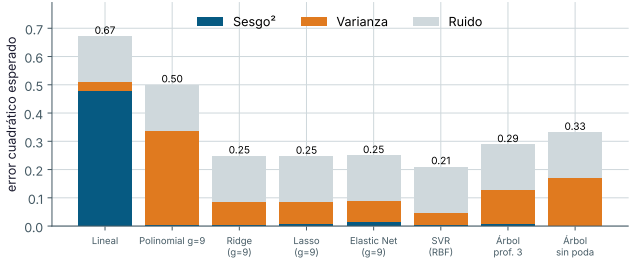
1. Lineal (OLS)
2. Polinomial de grado 9 (OLS)
3. Ridge sobre polinomio de grado 9
4. Lasso sobre polinomio de grado 9
5. Elastic Net sobre polinomio de grado 9
6. SVR con kernel RBF
7. Árbol de profundidad 3
8. Árbol sin poda

D3 · Lo que se ve: 30 de las 150 curvas ajustadas por cada modelo



Línea negra discontinua: la verdad. Línea gruesa: promedio de las 150 curvas. Curvas tenues: ajustes individuales. **Sesgo** = separación entre línea gruesa y discontinua; **varianza** = ancho del "manojo" de curvas.

D4 · Descomposición numérica del error



Lectura

- **Lineal**: casi todo es **sesgo** (0.479); varianza mínima.
- **Polinomio g=9**: sesgo ≈ 0 pero **varianza** 0.333.
- **Ridge, Lasso y Elastic Net** dejan el sesgo casi igual y **reducen la varianza** a $\approx$0.082.
- **SVR**: el menor error total (0.209); el mínimo posible es el ruido, 0.16.
- **Árboles**: sesgo bajo, varianza alta; podar baja la varianza (0.169 $\rightarrow$ 0.118).

D5 · Tabla resumen del experimento

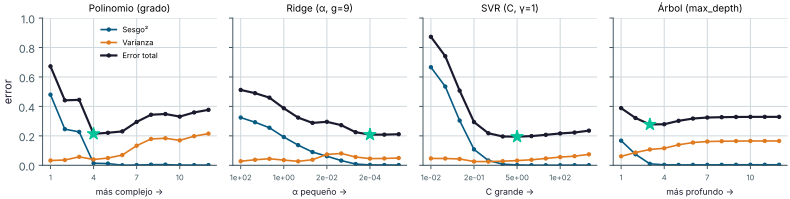
Error esperado = sesgo^2 + varianza + ruido (0.16)

Modelo	Sesgo^2	Varianza	Total
Lineal	0.479	0.031	0.670
Polinomial g=9	0.005	0.333	0.499
Ridge (g=9)	0.005	0.082	0.247
Lasso (g=9)	0.009	0.077	0.246
Elastic Net (g=9)	0.016	0.072	0.248
SVR (RBF)	0.004	0.045	0.209
Árbol prof. 3	0.009	0.118	0.287
Árbol sin poda	0.001	0.169	0.331

Precaución

Es **un solo problema**: una función suave en una variable con 40 datos. En otros datos el orden puede cambiar. Lo que **sí** generaliza es el **mecanismo**: cada familia tiene un control que mueve sesgo y varianza en sentidos contrarios.

D6 · El control de cada familia: la “U” en todos los modelos



Lectura

Sesgo^2 (azul) baja y varianza (naranja) sube al aumentar la complejidad; la **estrella** marca el mínimo del error total (grado 4; $\alpha \approx 2 \times 10^{-4}$; $C \approx 5$; profundidad 3).

Mismo patrón, cuatro familias

Error total mínimo: polinomio 0.21, SVR 0.19, árbol 0.28. Elegir ese control es **buscar la estrella** con validación cruzada.

D7 · Guía: ¿qué control muevo según el diagnóstico?

De síntomas a acciones (RMSE de entrenamiento y de validación)

Modelo	Sesgo alto (subajuste): complejizar	Varianza alta (sobreajuste): simplificar
Lineal / polinomial	subir el grado; agregar variables	bajar el grado; más datos
Ridge / Lasso / EN	bajar α	subir α (Lasso: elimina variables)
SVR	subir C o γ; bajar ε	bajar C o γ; subir ε
Árbol	subir `max_depth`; bajar `min_samples_leaf`	bajar `max_depth`; subir `min_samples_leaf`

Entrenamiento y validación **altos y parecidos** $\Rightarrow$ sesgo. Entrenamiento bajo y validación alta (**brecha**) $\Rightarrow$ varianza.

D8 · Comparar modelos de regresión con rigor

Protocolo para el informe comparativo

1. Mismo **conjunto de prueba** y mismas particiones para todos.
2. Cada modelo en un `Pipeline` (escalado dentro), con su **malla** de hiperparámetros.
3. **Validación cruzada anidada** para comparar: media $\pm$ desviación del RMSE.
4. Diagnóstico de cada uno: brecha entrenamiento–validación (sesgo o varianza) y curva de validación de su control.
5. Entrenar los finalistas con todo el entrenamiento y medir en prueba **una vez**.

Qué reportar

- Tabla: modelo, hiperparámetros elegidos, RMSE, MAE y R^2 (media $\pm$ desv.).
- Gráfica de residuos del mejor modelo.
- Una explicación **en términos de sesgo y varianza** de por qué ganó (y cuánto mejor es: ¿diferencia mayor que la dispersión?).
- Interpretabilidad y costo de cómputo como criterios de desempate.

Pausa para pensar (actividad 4, 3 min)

Trabaja de forma individual

Tres modelos dan RMSE (entrenamiento / validación): **A**: 2.9 / 3.0. **B**: 0.2 / 2.8. **C**: 1.1 / 1.4. El ruido irreducible es 1.0.

1. ¿Cuál sufre sesgo alto y cuál varianza alta?
2. ¿Cuál es el mejor modelo? ¿Qué le harías a los otros dos?
3. Si B es un árbol, ¿qué dos hiperparámetros ajustarías?

Respuestas: (1) A: sesgo alto (ambos errores altos y juntos); B: varianza alta (gran brecha). (2) C: cercano al ruido y sin brecha. A: más complejidad (grado, menos α, más profundidad). B: simplificar (más α, menos profundidad). (3) Bajar `max_depth` y subir `min_samples_leaf`.

Resumen de la sesión

Ideas clave

- **CART**: preguntas de sí o no, elegidas de forma voraz para **minimizar el MSE ponderado** de los hijos; la hoja predice el promedio.
- **Poda**: `max_depth` y `min_samples_leaf` (más `ccp_alpha`) controlan la complejidad; sin ellas el árbol memoriza.
- Los árboles son legibles pero **inestables** y **no extrapolan**.

Para llevarse

- **Validación cruzada**: $k = 5$ o 10; **estratificada** si hay desbalance (o y muy asimétrico); **anidada** para comparar modelos sin optimismo.
- **Sesgo–varianza**: cada familia tiene su control; lineal = sesgo, polinomio y árbol sin poda = varianza, regularizar y podar = el punto medio.
- Todo preprocesamiento **dentro del** `Pipeline`; prueba **una sola vez**.

Siguiente sesión

Módulo 3: regresión logística y kNN para clasificación (sesión 8).

Actividad de laboratorio (individual)

Repaso de medio curso + informe comparativo de modelos de regresión

Cuaderno de Jupyter sobre un conjunto de datos de regresión del curso. Trabaja **de forma individual**.

1. Divide en entrenamiento y prueba; construye un `Pipeline` para cada modelo (lineal, polinomial, Ridge, Lasso, Elastic Net, SVR y árbol).
2. Ajusta el **árbol**: curva de validación de `max_depth` y malla con `min_samples_leaf`; dibuja el árbol podado.
3. Ajusta los hiperparámetros del resto con `GridSearchCV`.
4. Compara todos con **validación cruzada anidada** (media $\pm$ desviación del RMSE).
5. Diagnostica cada modelo con curvas de aprendizaje: ¿sesgo o varianza?
6. Mide el mejor en el conjunto de prueba **una sola vez**; grafica residuos.
7. Redacta un informe breve con tabla y conclusiones.

Entregable: el cuaderno con el informe (¿qué modelo ganó y por qué en términos de sesgo y varianza?, ¿la diferencia supera la dispersión?).



¿Preguntas?

¡Pasemos al repaso de medio curso y al informe comparativo de modelos de regresión!